

Code Explanation

E2E Address Change YUYU Processing Code Explanation

This code is designed to process address changes using Apache Spark, reading data from Oracle tables, applying transformations, and writing the output to a CSV file while updating a staging table with processing metadata.

Overview

The code is structured into several functions, each responsible for a specific task in the data processing pipeline. It starts by creating a Spark session, reading source data from Oracle tables, applying transformations to the data, writing the transformed data to an output CSV file, and updating a staging table with processing metadata.

Step 1: Creating a Spark Session

The first step is to create a Spark session, which is the entry point to any Spark functionality. The `create_spark_session` function configures a Spark session with the necessary settings for Delta Lake support.

```
def create_spark_session() -> SparkSession:
    """Create and configure a Spark session"""
    return (SparkSession.builder
            .appName("E2E Address Change YUYU Processing")
            .config("spark.sql.extensions", "io.delta.sql.DeltaSpark
SessionExtension")
            .config("spark.sql.catalog.spark_catalog", "org.apache.s
park.sql.delta.catalog.DeltaCatalog")
            .getOrCreate())
```

Step 2: Reading Source Data from Oracle Tables

The `read_source_data` function reads data from three Oracle tables: the main source table and two lookup tables. It uses the `jdbc` format to connect to Oracle and reads the data into DataFrames.

```
def read_source_data(spark: SparkSession) -> tuple:
    # Read main source table
    source_df = (spark.read
                .format("jdbc")
                .option("url", JDBC_URL)
                .option("dbtable", SOURCE_TABLE)
                .option("user", JDBC_PROPERTIES["user"])
                .option("password", JDBC_PROPERTIES["password"])
                .option("driver", JDBC_PROPERTIES["driver"])
                .schema(STG_E2E_AC_TXDBH_DATA_SCHEMA)
                .load())

    # Read lookup tables
    yuyu_clnt_df = (spark.read
                   .format("jdbc")
                   .option("url", JDBC_URL)
                   .option("dbtable", YUYU_CLNT_TABLE)
                   .option("user", JDBC_PROPERTIES["user"])
                   .option("password", JDBC_PROPERTIES["password"])
                   .option("driver", JDBC_PROPERTIES["driver"])
```

```

        .schema(YUYU_CLNT_SCHEMA)
        .load())

yuyuk_cln_df = (spark.read
                .format("jdbc")
                .option("url", JDBC_URL)
                .option("dbtable", YUYUK_CLN_TABLE)
                .option("user", JDBC_PROPERTIES["user"])
                .option("password", JDBC_PROPERTIES["password"])
                .option("driver", JDBC_PROPERTIES["driver"])
                .schema(YUYUK_CLN_SCHEMA)
                .load())

return source_df, yuyu_clnt_df, yuyuk_cln_df

```

Step 3: Applying Transformations to the Data

The `apply\_address\_change\_transformations` function applies the necessary transformations to the source data and lookup data. The implementation of this function is not shown in the provided code snippet, but it returns two DataFrames: `yuyu\_output\_df` and `staging\_update\_df`.

```

yuyu_output_df, staging_update_df = apply_address_change_transformations(
    source_df, yuyu_clnt_df, yuyuk_cln_df, PROCESS_USERID
)

```

Step 4: Writing the Output to a CSV File

The `write\_output\_file` function writes the transformed data (`yuyu\_output\_df`) to a CSV file at the specified `OUTPUT\_FILE\_PATH`.

```

def write_output_file(yuyu_output_df, output_path):
    """Write the output file in the required format"""
    (yuyu_output_df
     .coalesce(1) # Ensure a single output file
     .write
     .mode("overwrite")
     .option("header", "true")
     .option("encoding", "UTF-8") # Ensure proper encoding for Japanese characters
     .csv(output_path))

```

Step 5: Updating the Staging Table

The `update\_staging\_table` function updates the staging table with processing metadata. It collects the update data to the driver, creates a JDBC connection, prepares an UPDATE statement, and executes it in batch mode.

```

def update_staging_table(staging_update_df, jdbc_url, jdbc_properties, target_table):
    # For each row in the staging_update_df, we need to construct an
    # execute an UPDATE statement
    # Since DataFrame.write doesn't support UPDATE operations directly

```

ly, we'll use JDBC batch updates

```
# Collect the update data to the driver (assuming the dataset is
not too large)
update_rows = staging_update_df.collect()

# Import required Java classes for JDBC operations
from pyspark.sql import SparkSession
spark = SparkSession.getActiveSession()

# Get a JDBC connection
connection = spark._jvm.java.sql.DriverManager.getConnection(
    jdbc_url,
    jdbc_properties["user"],
    jdbc_properties["password"]
)

try:
    # Prepare the UPDATE statement
    update_sql = """
    UPDATE STG_E2E_AC_TXDBH_DATA
    SET STG_PROCESS_DATE = ?,
        STG_PROCESS_USERID = ?
    WHERE T_TX_REQUEST_REQUEST_ID = ?
        AND T_TX_BASIC_TRANS_ID = ?
        AND T_TX_RELATION_TRANS_REL_ID = ?
        AND T_TX_REQ_POL_REQUEST_POLICY_ID = ?
        AND STG_POLICY_ID = ?
    """

    # Create a prepared statement
    prepared_stmt = connection.prepareStatement(update_sql)

    # Set auto-commit to false for batch processing
    connection.setAutoCommit(False)

    # Add batch updates
    for row in update_rows:
        prepared_stmt.setTimestamp(1, row["STG_PROCESS_DATE"])
        prepared_stmt.setString(2, row["STG_PROCESS_USERID"])
        prepared_stmt.setString(3, row["T_TX_REQUEST_REQUEST_ID"
ID"])
        prepared_stmt.setString(4, row["T_TX_BASIC_TRANS_ID"])
        prepared_stmt.setString(5, row["T_TX_RELATION_TRANS_REL_
ID"])
        prepared_stmt.setString(6, row["T_TX_REQ_POL_REQUEST_POL
ICY_ID"])
        prepared_stmt.setString(7, row["STG_POLICY_ID"])
        prepared_stmt.addBatch()

    # Execute the batch update
    update_counts = prepared_stmt.executeBatch()

    # Commit the transaction
    connection.commit()

    # Log the number of updated records
    total_updated = sum(update_counts)
    print(f"Updated {total_updated} records in {target_table}")
```

```

except Exception as e:
    # Roll back in case of error
    connection.rollback()
    raise e
finally:
    # Close resources
    if 'prepared_stmt' in locals():
        prepared_stmt.close()
    connection.close()

```

Step 6: Main Processing Function

The `main` function orchestrates the entire data processing pipeline by calling the above functions in sequence. It creates a Spark session, reads source data, applies transformations, writes the output to a CSV file, and updates the staging table.

```

def main():
    """Main processing function"""
    # Create Spark session
    spark = create_spark_session()

    try:
        # Read source data
        source_df, yuyu_clnt_df, yuyuk_cln_df = read_source_data(spark)

        # Apply transformations
        yuyu_output_df, staging_update_df = apply_address_change_transformations(
            source_df, yuyu_clnt_df, yuyuk_cln_df, PROCESS_USERID
        )

        # Write output file
        write_output_file(yuyu_output_df, OUTPUT_FILE_PATH)

        # Update staging table
        update_staging_table(staging_update_df, JDBC_URL, JDBC_PROPERTIES, SOURCE_TABLE)

        print("E2E Address Change YUYU processing completed successfully")

    except Exception as e:
        print(f"Error in E2E Address Change YUYU processing: {str(e)}")
        raise
    finally:
        spark.stop()

```